Understanding Ingress in Kubernetes: Key Concepts and Configuration

Introduction:

Ingress in Kubernetes is a vital resource for managing external access to services within a cluster. It provides a streamlined way to route HTTP and HTTPS traffic to various services based on criteria such as URL paths and hostnames. By using Ingress, you can consolidate multiple services under a single external IP address, simplifying traffic management and reducing the need for multiple Load Balancers or NodePorts. This guide will walk you through the essential concepts of Ingress, its benefits, and how to set up an NGINX Ingress Controller to handle traffic routing effectively. You will learn about different types of Ingress rules, such as path-based and host-based routing, and how to configure SSL termination for secure connections.

This is a demonstration of how to use Kubernetes Ingress to route traffic to different services in your cluster based on different paths.

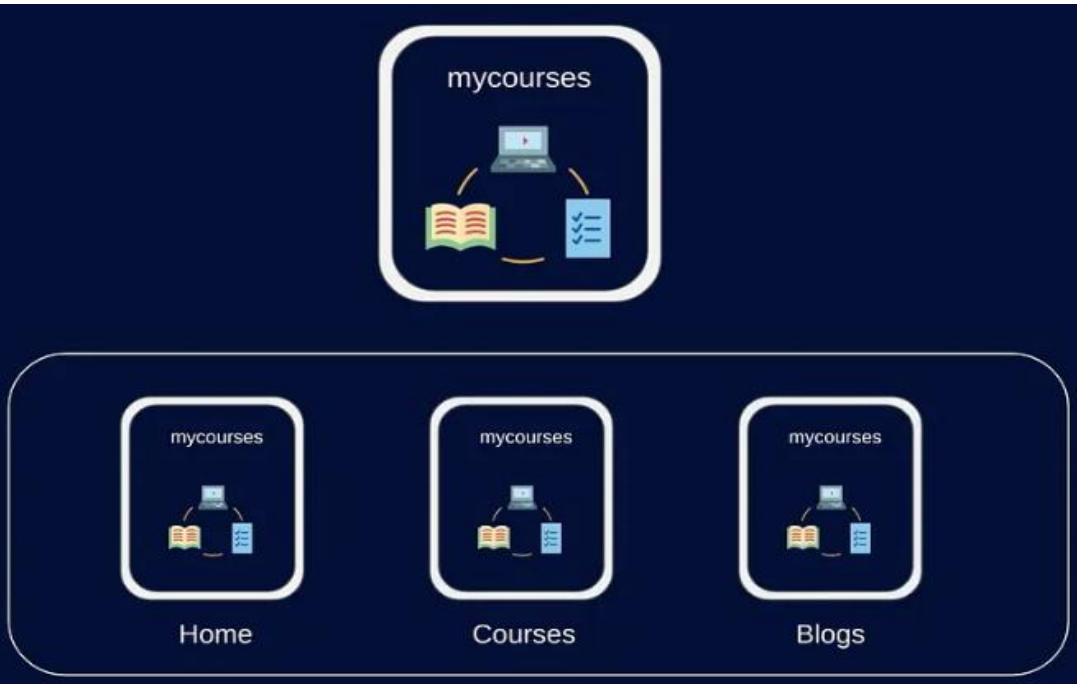
What is Ingress?

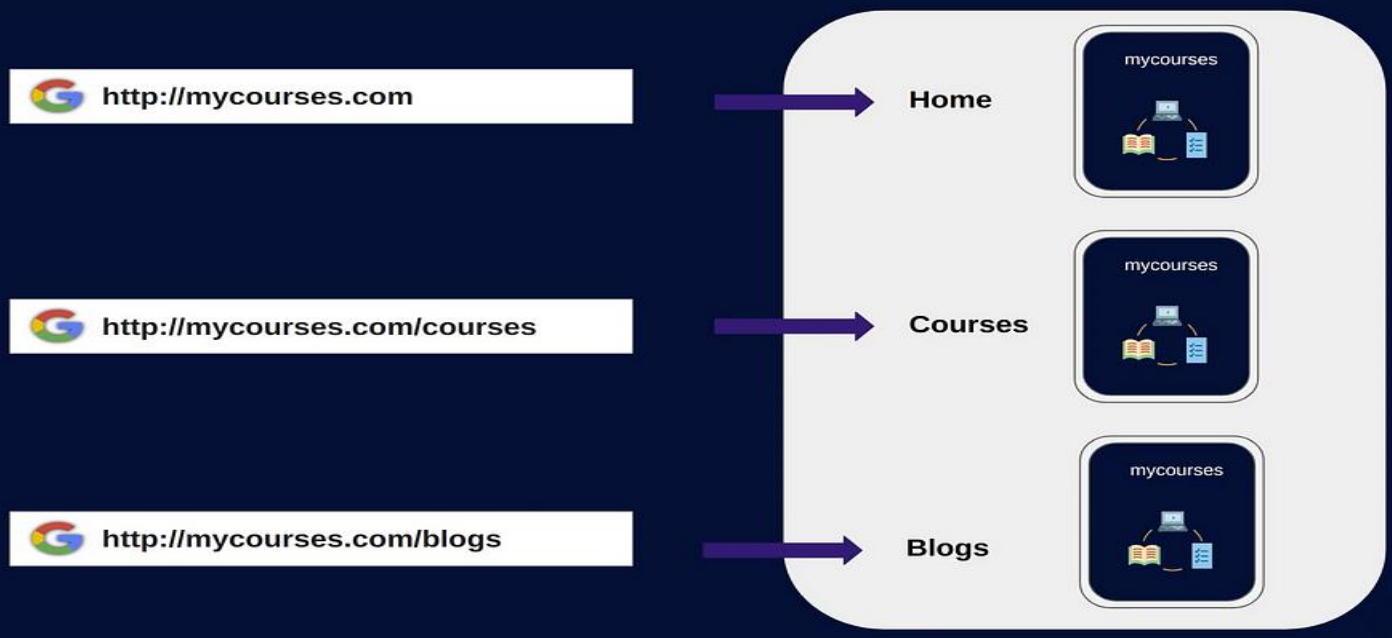
Ingress in Kubernetes is a resource that manages external access to services within a cluster, typically HTTP or HTTPS traffic. It allows you to define rules for routing traffic to different services based on URL paths, hostnames, or other criteria, without exposing multiple services to the outside world directly. Essentially, Ingress acts as a gateway that directs incoming requests to the appropriate services within the cluster.

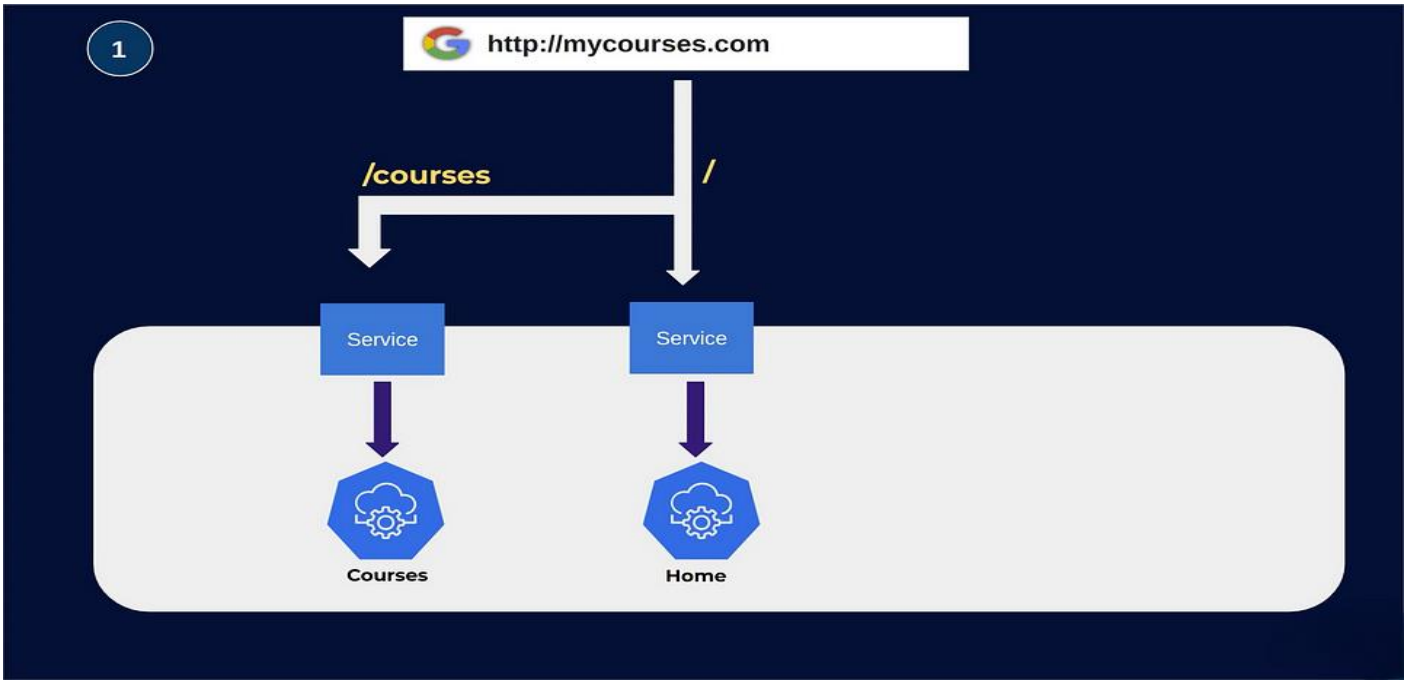
Why Ingress?

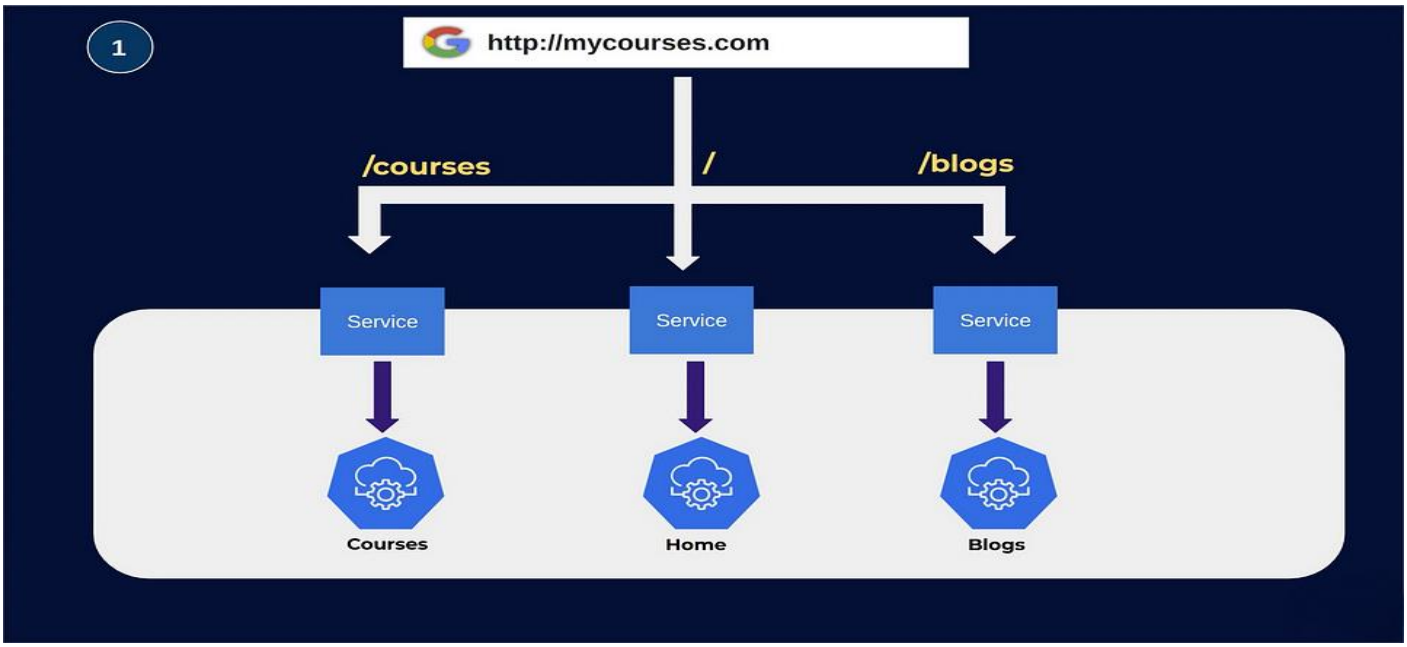
Imagine you run multiple web applications (e.g., a blog, an e-commerce site, and an API) on the same Kubernetes cluster. Without Ingress, you'd need to expose each service with its own LoadBalancer or NodePort, which can be expensive and complex to manage. Ingress simplifies this by allowing you to route traffic to the correct service based on the URL path or hostname, all under a single external IP address.

DevOps Engineer





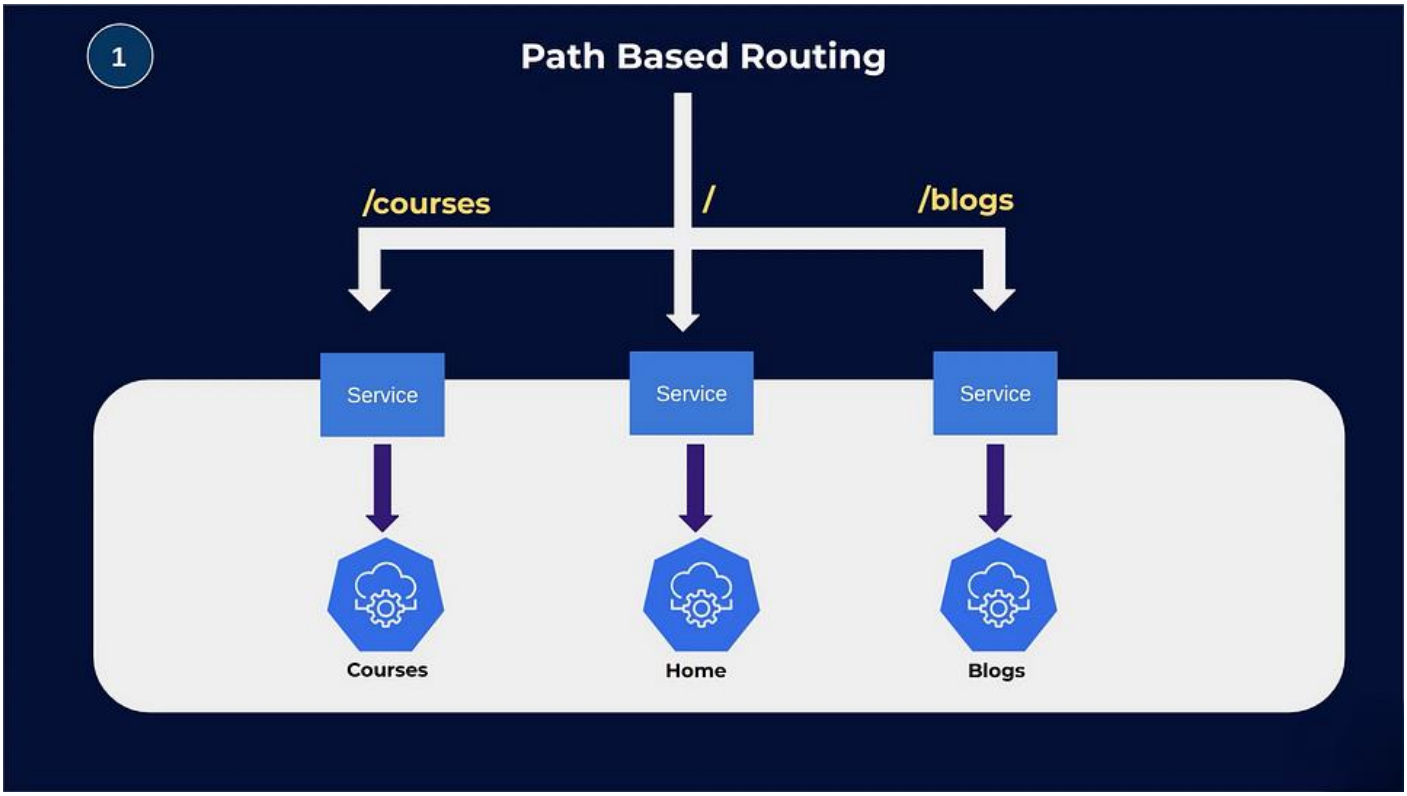




Types of Ingress Rules:

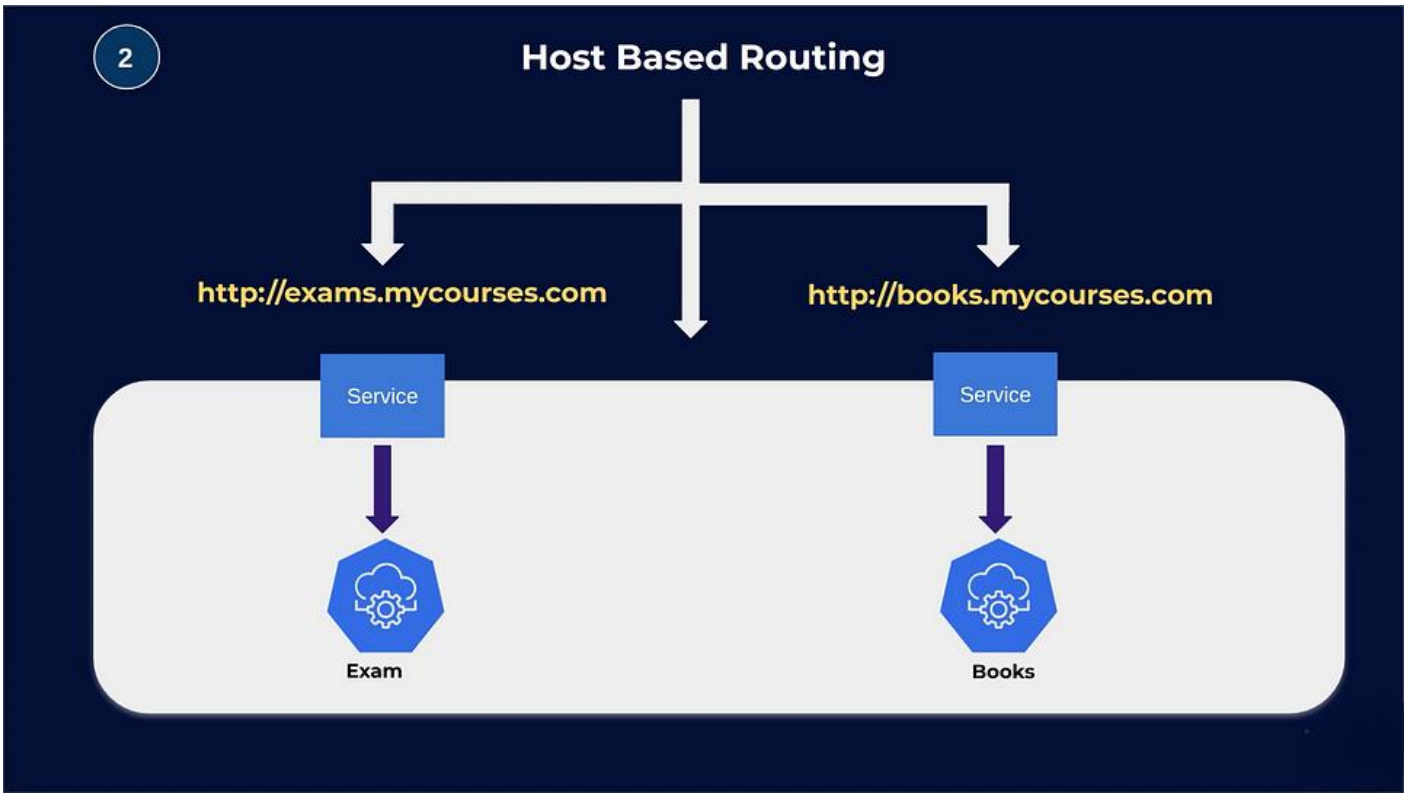
1. **Path-Based Routing**: Routes traffic to different services based on the URL path.

- **Example**: If you run an e-commerce site, traffic to yourdomain.com/courses might go to the shopping service, while traffic to yourdomain.com/blogs goes to the blog service.



2. Host-Based Routing: Routes traffic based on the hostname.

- **Example**: Traffic to https://exams.mycourses.com goes to your shopping service, while traffic to https://books.mycourses.com goes to your blog service.



3. SSL Termination: Handles HTTPS traffic by terminating SSL/TLS at the Ingress, meaning the Ingress manages the encryption and forwards the traffic to services as plain HTTP.

- **Example**: You can secure your services with HTTPS by managing certificates at the Ingress level, offloading the complexity of SSL management from the individual services.

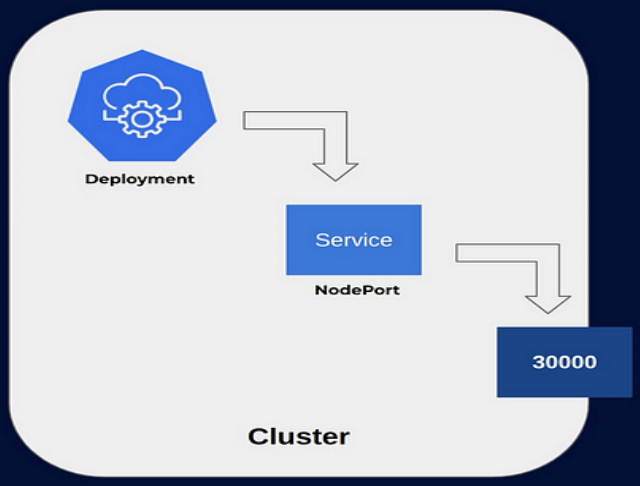
3

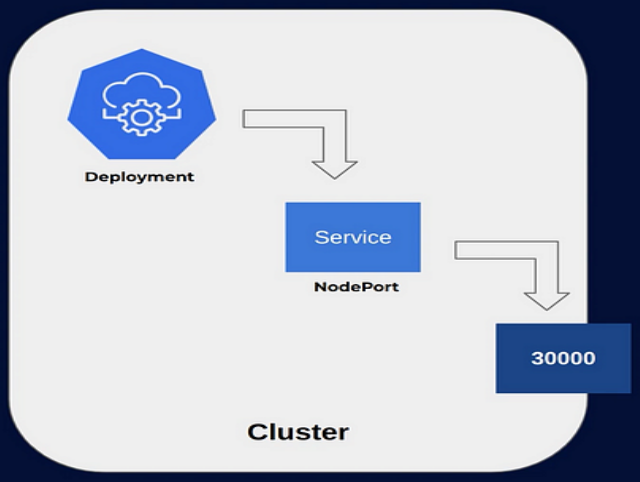
SSL termination

https://exams.mycourses.com

https://books.mycourses.com

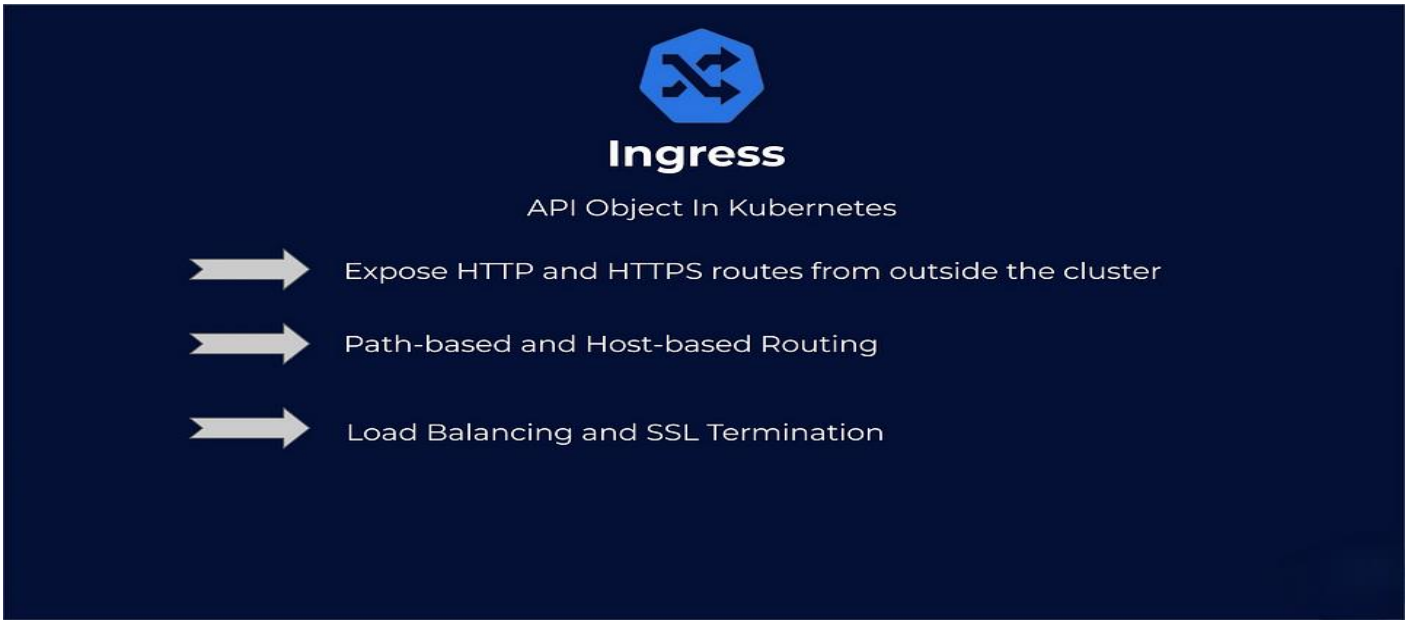
https://mycourses.com





http://IP:Port

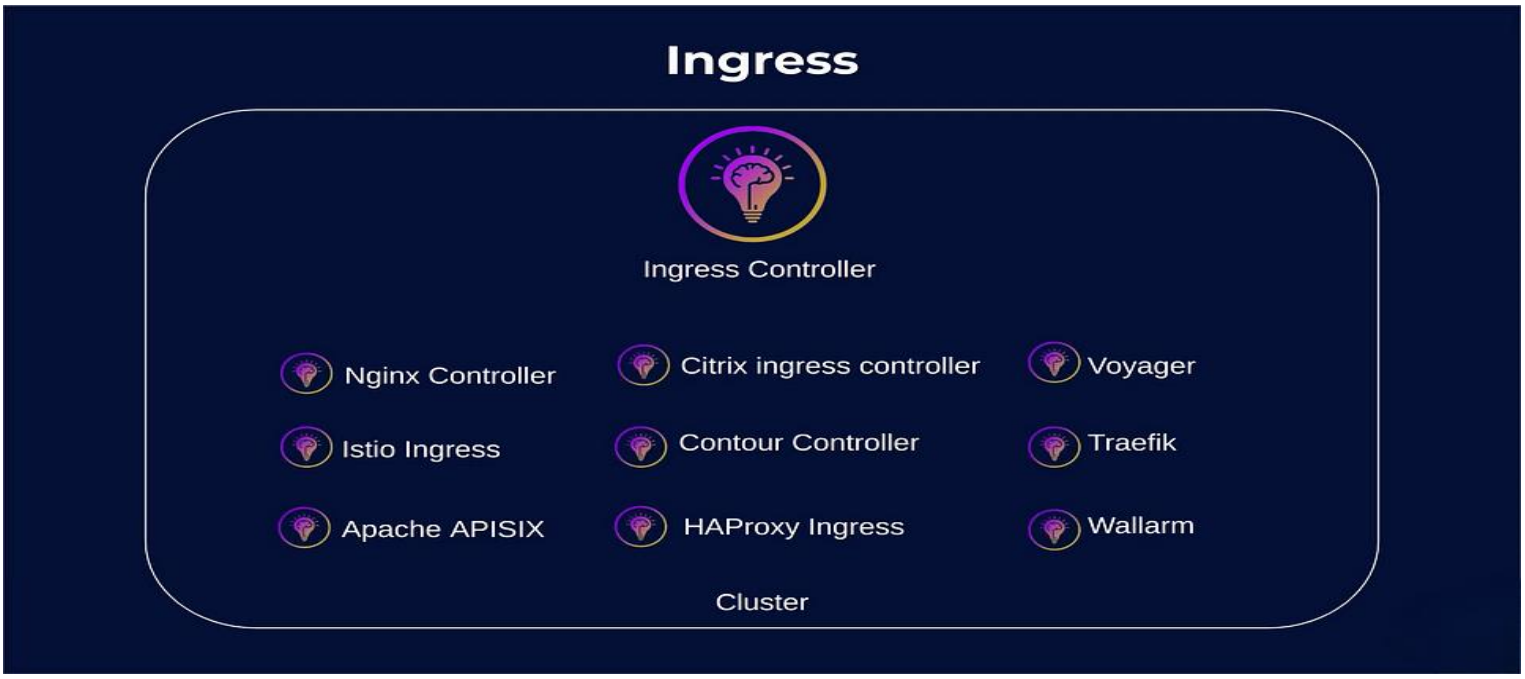
http://67.23.45.67:30000



Types of Ingress Controllers:

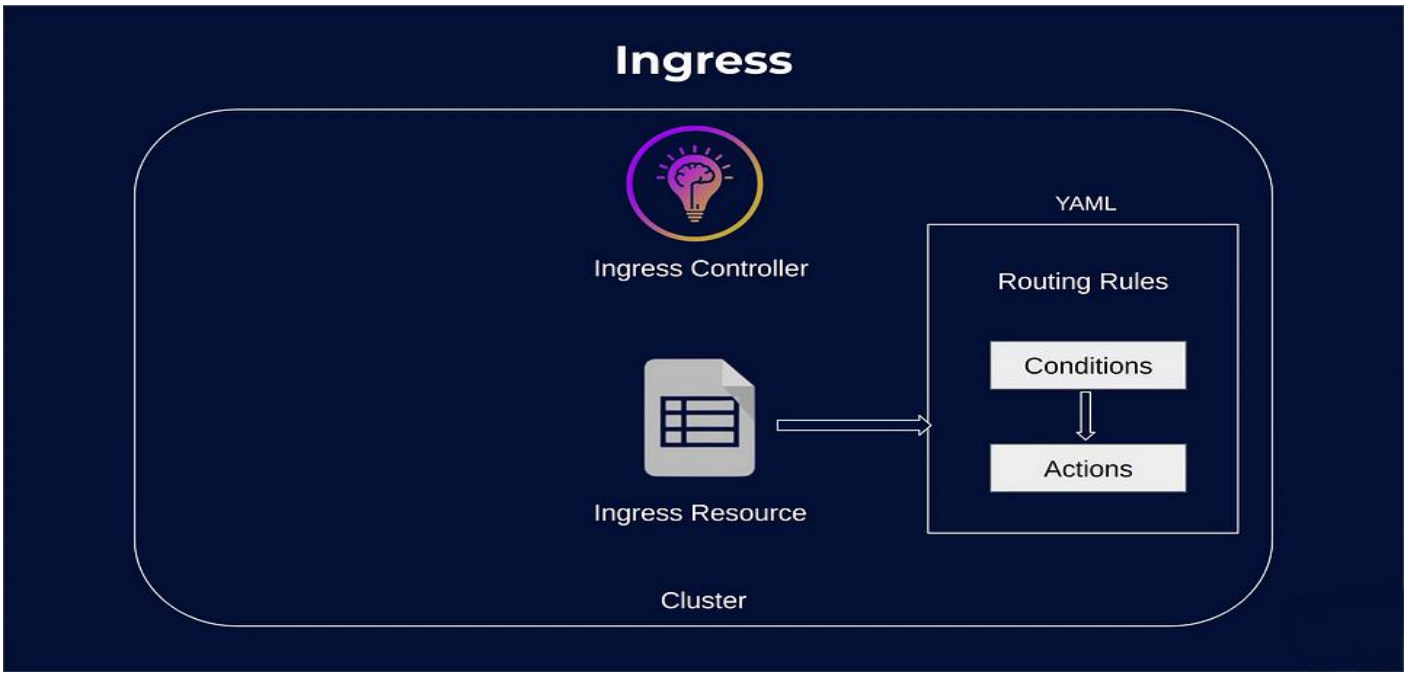
Ingress controllers are implementations that fulfill the Ingress resource and manage the routing of traffic. Some popular Ingress controllers include:

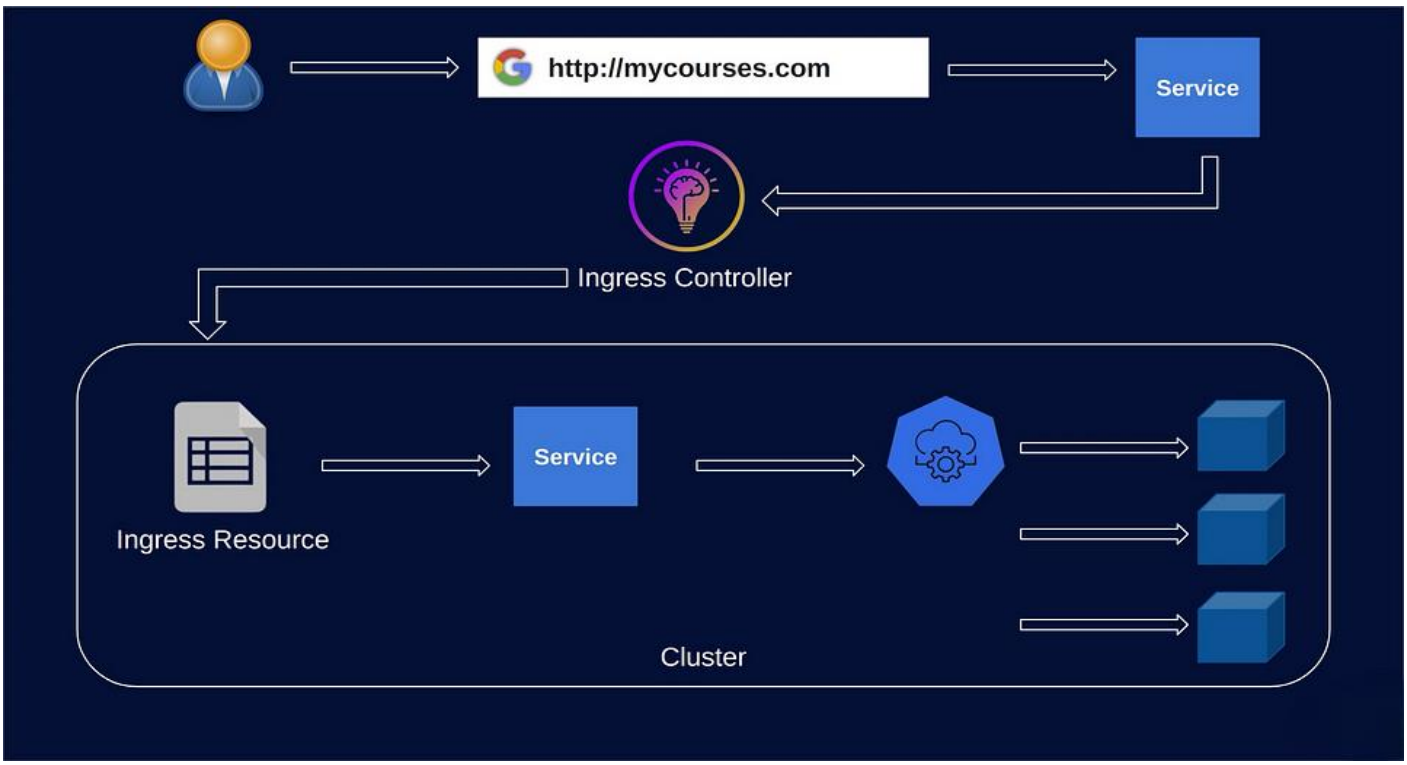
1. **NGINX Ingress Controller**: A popular, widely used controller that integrates well with Kubernetes and provides robust features for load balancing, SSL termination, and routing.
2. **HAProxy Ingress Controller**: Known for its high performance and reliability, HAProxy is used for load balancing and can handle complex routing scenarios.
3. **Istio Ingress Gateway**: Part of the Istio service mesh, it provides advanced traffic management, security, and observability features beyond standard Ingress controllers.
4. **Contour**: A Kubernetes-native ingress controller that uses Envoy as the data plane, known for its fast performance and dynamic configuration.
5. **Citrix Ingress Controller**: Offers enterprise-grade load balancing and security features, with integration into Citrix ADC.
6. **Apache APISIX Ingress Controller**: An open-source API Gateway that also functions as an ingress controller, offering rich plugins and high performance.



How It Works?

Ingress
Ingress Controller
Ingress Resource
Cluster


Ingress
Ingress Controller
YAML
Routing Rules
Conditions
Actions
Ingress Resource
Cluster


http://mycourses.com
Service
Ingress Controller
Ingress Resource
Service
Cluster

1. User Request :-
A user tries to access a service within the Kubernetes cluster by entering a URL, like http://mycourses.com, in their web browser.
2. DNS Resolution:-
The URL is resolved to an external IP address associated with the Ingress Controller. This is usually handled by DNS.
3. Ingress Controller:-
The Ingress Controller is a component running within the Kubernetes cluster. It monitors the Ingress resources and manages the routing rules for incoming traffic.
Upon receiving the request, the Ingress Controller evaluates the URL against the Ingress rules defined in the Ingress Resource.
4. Ingress Resource:-
The Ingress Resource is a YAML configuration file that defines the routing rules for different paths and hosts.
It specifies how requests should be directed to the appropriate services within the cluster. For example, it might route http://mycourses.com/exam to the "shop" service and http://mycourses.com/blogs to the "blog" service.
5. Routing Traffic to Services:-
The Ingress Controller forwards the request to the appropriate service based on the rules defined in the Ingress Resource.
The service then routes the request to the correct pod(s) running the application within the cluster.
6. Service Response:-
The service processes the request and returns the response to the Ingress Controller.
7. Response to the User:-
Finally, the Ingress Controller sends the response back to the user, displaying the requested content in their web browser.
Summary:-
Ingress Controller: Acts as a gateway that processes external requests and directs them to the appropriate services based on Ingress rules.
Ingress Resource: Defines how traffic should be routed to services within the cluster.
Service: Handles the actual application logic and responds to the user request.
Pods: The units of the application that are managed by the services and run inside the cluster.
This setup allows multiple services to be exposed via a single external IP address, simplifying the management of traffic to various applications running within a Kubernetes cluster.

Ingress Resource File

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
spec:
  rules:
  - host: "mycourses.com"
    http:
      paths:
      - pathType: Prefix
        path: /
        backend:
          service:
            name: home-service
            port:
              number: 8080
      - pathType: Prefix
        path: /courses
        backend:
          service:
            name: course-service
            port:
              number: 8181
```

nstall nginx-ingress controller :
To get started, you will need to install the nginx-ingress controller in your Kubernetes cluster by running the following command:
“kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml”
This will deploy the nginx-ingress controller as a Deployment in your cluster.

```
root@ip-172-31-37-201:~# kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/
deploy.yaml
namespace/ingress-nginx created
serviceaccount/ingress-nginx created
serviceaccount/ingress-nginx-admission created
role.rbac.authorization.k8s.io/ingress-nginx created
role.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrole.rbac.authorization.k8s.io/ingress-nginx created
clusterrole.rbac.authorization.k8s.io/ingress-nginx-admission created
rolebinding.rbac.authorization.k8s.io/ingress-nginx created
rolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
configmap/ingress-nginx-controller created
service/ingress-nginx-controller created
service/ingress-nginx-controller-admission created
deployment.apps/ingress-nginx-controller created
job.batch/ingress-nginx-admission-create created
job.batch/ingress-nginx-admission-patch created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/ingress-nginx-admission created
```

“kubectl get ns”

```
root@ip-172-31-37-201:~# kubectl get ns
NAME                   STATUS   AGE
default                Active   33h
ingress-nginx          Active   8s
kube-node-lease        Active   33h
kube-public            Active   33h
kube-system            Active   33h
local-path-storage     Active   33h
```

“kubectl get all -n ingress-nginx”

```
root@ip-172-31-37-201:~# kubectl get all -n ingress-nginx
NAME                                        READY   STATUS      RESTARTS   AGE
pod/ingress-nginx-admission-create-wj4k2    0/1     Completed   0          2m3s
pod/ingress-nginx-admission-patch-l2mt2     0/1     Completed   0          2m3s
pod/ingress-nginx-controller-d45d995d4-rfs8l   1/1     Running     0          2m3s

NAME                                         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
service/ingress-nginx-controller             NodePort    10.96.135.106   <none>        80:32223/TCP,443:31810/TCP   2m3s
service/ingress-nginx-controller-admission   ClusterIP   10.96.237.126   <none>        443/TCP                      2m3s

NAME                                       READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/ingress-nginx-controller   1/1     1            1           2m3s

NAME                                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/ingress-nginx-controller-d45d995d4    1         1         1       2m3s

NAME                                       STATUS     COMPLETIONS   DURATION   AGE
job.batch/ingress-nginx-admission-create   Complete   1/1           9s         2m3s
job.batch/ingress-nginx-admission-patch    Complete   1/1           9s         2m3s
```

Create Deployments

Next, you will need to create some sample Deployments to route traffic to. Run the following commands to create the Deployments:

“kubectl create deploy sample-1 --image=devopsprosamples/next-path-sample-1
kubectl create deploy sample-2 --image=devopsprosamples/next-path-sample-2
kubectl create deploy sample-3 --image=devopsprosamples/next-sample-1
kubectl create deploy sample-4 --image=devopsprosamples/next-sample-2”

These commands will create four sample Deployments with different images.

```
root@ip-172-31-37-201:~# kubectl create deploy sample-1 --image=devopsprosamples/next-path-sample-1
kubectl create deploy sample-2 --image=devopsprosamples/next-path-sample-2
kubectl create deploy sample-3 --image=devopsprosamples/next-sample-1
kubectl create deploy sample-4 --image=devopsprosamples/next-sample-2
deployment.apps/sample-1 created
deployment.apps/sample-2 created
deployment.apps/sample-3 created
deployment.apps/sample-4 created
```

“kubectl get po”

```
root@ip-172-31-37-201:~# kubectl get po
NAME                      READY   STATUS    RESTARTS   AGE
sample-1-85d9475b56-kfbk7   1/1     Running   0          24s
sample-2-854ccddb9f-p2927   1/1     Running   0          24s
sample-3-64c76c4544-8vxlz   1/1     Running   0          24s
sample-4-57bcb8c55c-25fr7   1/1     Running   0          24s
secret-demo-1               1/1     Running   0          73m
secret-demo-2               1/1     Running   0          48m
```

Create services

After you have created the Deployments, you will need to create Services for each of them. Run the following commands to create the Services:

“kubectl expose deploy sample-1 --type=ClusterIP --port=3000
kubectl expose deploy sample-2 --type=ClusterIP --port=3000
kubectl expose deploy sample-3 --type=ClusterIP --port=3000
kubectl expose deploy sample-4 --type=ClusterIP --port=3000”

These commands will create four Services with ClusterIP type for each of the sample Deployments.

Create Ingress resource:-

Now that you have created the Services, you can create an Ingress resource to route traffic to them based on different paths. To create the Ingress resource, run the following command:

ingress-resource.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: nginx
  tls:
  - hosts:
      - "test.local"
    secretName: tls-secret
  rules:
  - host: "test.local"
    http:
      paths:
      - pathType: Prefix
        path: /sample-1
        backend:
          service:
            name: sample-1
            port:
              number: 3000
      - pathType: Prefix
        path: /sample-2
        backend:
          service:
            name: sample-2
            port:
              number: 3000
      - pathType: Prefix
        path: /sample-3
        backend:
          service:
            name: sample-3
            port:
              number: 3000
      - pathType: Prefix
        path: /sample-4
        backend:
          service:
            name: sample-4
            port:
              number: 3000
```

“kubectl apply -f ingress-resource.yaml”

This will create an Ingress resource with rules to route traffic to the sample Services based on different paths.